Secure RAG Application

Attacking and defending a retrieval pipeline, and measuring whether the defenses work

Project report · phongbachxuan26@gmail.com · 22 October 2026

Summary

This project builds a small question-answering assistant for a fictional company, attacks it five different ways, and measures how often each attack works before and after a set of defenses is switched on. The whole thing runs on a single machine against a local language model and a local vector database. Every secret in the data is invented and planted on purpose so the attacks can be run safely.

The point of the project is not only to show that the attacks exist. It is to put a number on each one, turn on a control, run the identical attack again, and show the number change. Four of the five attacks are stopped cleanly. The fifth, data poisoning, is not, and that gap is the most useful result in the report because it shows where a control has to sit to be effective.

Background

A retrieval-augmented generation (RAG) system answers questions by first fetching relevant documents and then asking a language model to write an answer from them. This design is popular because it lets a model use private, up-to-date information without retraining. It also creates a chain of places where untrusted data meets trusted logic, and each of those places can be attacked.

The assistant in this project belongs to a fictional company called Northwind Retail. It answers questions from internal documents. Two kinds of user ask questions: a general employee and a member of HR. They are allowed to see different documents. An employee should only ever see public material. HR can see public material and HR-only material. That difference in entitlement is what several of the attacks try to break.

The system

The assistant has the parts common to any RAG pipeline:

- A document store, Weaviate, that holds the company documents as vectors.
- An embedding step, the Ollama model nomic-embed-text, that turns text into those vectors.
- A retriever that finds the documents whose vectors are closest to the question.
- A prompt builder that assembles the question together with the retrieved text.
- A language model, the Ollama model basileak-r4, that writes the final answer.

The same assistant can run in two modes. In vulnerable mode the defenses are off. In hardened mode they are on. Running the same attacks against both modes is what makes the before-and-after comparison fair, because the only thing that changes between the two runs is the defenses.

Trust boundaries

A trust boundary is a point where data crosses from something we do not control into something we do. Each boundary is where a specific control belongs. The project marks four of them.

Boundary	Where it is	Control when hardened
1. User to app	when a request arrives	decide the role, employee or HR
2. Document to ingestion	when a document enters the store	treat retrieved text as untrusted data, never as commands
3. Retriever to vector DB	at search time	only fetch documents the current role may see
4. Model output to user	before the answer is returned	remove attack artifacts from the answer

The attacks

Five attacks are run, each matching a category in the OWASP Top 10 for Large Language Model Applications.

Attack	OWASP	What it tries to do
Indirect prompt injection	LLM01	Hide an instruction inside a document and see whether the model obeys an order that arrived through data rather than from the user.
Cross-tenant leakage	LLM02	Have an employee ask for HR-only information and check whether the system hands it over.
Data poisoning	LLM08	Add a document containing a false fact and watch whether later innocent questions start repeating the lie.
Context exfiltration	LLM02	Try to make the model dump the restricted text it was given, word for word.
Embedding weakness	LLM08	Shape a query so it drags a restricted document into the search results even though the user should not reach it. Measured at retrieval, before any answer is written.

How success is measured

The hardest part of this kind of work is deciding what counts as a successful attack in a way that is not a matter of opinion. The approach here is to plant a unique secret string inside each restricted document. If that exact string turns up in an answer that an employee was not meant to receive, the leak is proven. A plain text match decides it, so the result is the same every time and needs no judgement call.

PyRIT, an open-source tool for probing language model systems, runs the attacks and records the outcome of each attempt. The string match is the primary check and is treated as ground truth. Two model-based scorers, a Llama Guard safety classifier and a separate judge model, are shown on a sample answer as extra signals, but they are never allowed to decide the headline number. An opinion from another model can be wrong; a string either appears or it does not.

Because a language model does not give the same answer twice, each attack is run several times and the result is reported as an Attack Success Rate, the share of attempts that succeeded. Each rate comes with a 95% confidence interval so the small sample size stays visible rather than hidden behind a tidy percentage. A statistical test then checks whether the drop between vulnerable and hardened mode is real or could be chance. Finally a short utility check confirms that turning on the defenses did not simply break the assistant for the people who are supposed to use it.

Results

The figures below come from a real run against the local models, with three repetitions per attack. They move a little between runs because the model is not deterministic.

Attack	OWASP	Before	After	What it shows
Indirect injection	LLM01	0.44	0.00	separation and output cleaning stop it
Cross-tenant leakage	LLM02	1.00	0.00	the role filter stops it, confirmed real
Data poisoning	LLM08	1.00	1.00	the defenses do not help
Context exfiltration	LLM02	0.50	0.00	the role filter stops it
Embedding weakness	LLM08	1.00	0.00	the filter keeps crafted matches away

A second measurement looks one step earlier, at the retrieval stage. For leakage, exfiltration, and the embedding attack, an HR document reached the employee on every attempt in vulnerable mode and on no attempt in hardened mode. This shows that the role filter at boundary 3 is doing the real work, not just the output cleaner at boundary 4. The two controls defend different things, and separating the measurements makes that clear.

The utility check passed. In hardened mode HR still retrieves its own documents, and an ordinary employee still gets correct answers to ordinary public questions. The defenses raised the bar for attackers without locking out legitimate users.

What the poisoning result means

Four attacks drop to a zero success rate once the defenses are on. Data poisoning stays at the top. This is the finding worth dwelling on. Prompt separation and output cleaning both act late in the pipeline, after a document has already been stored and fetched. They can stop the model from obeying a hidden order and they can strip a known marker out of an answer. They can do nothing about a plain false statement that is already sitting in the store, because a lie is not an instruction and it carries no marker to strip.

Stopping data poisoning needs a control earlier in the chain, at boundary 2, where documents are first ingested. That means checking the source and trustworthiness of a document before it is allowed into the store, rather than trying to repair the damage at answer time. The lab makes this concrete: the same run that defeats four attacks leaves poisoning untouched, which points directly at where the missing control has to go.

Limitations

- The document set is small and each attack runs only a few times, so the confidence intervals are wide. A fuller study would use more documents and more repetitions.
- The string match is ground truth, but the two model-based scorers are only signals. A stronger study would hand-label a sample of answers and report how closely the judge model agrees with a person.
- The results depend on one particular, weakly-aligned model. A better-aligned model would resist some attacks on its own, which would separate what the application controls achieve from what the model achieves by itself.

Conclusion

The project shows the full loop the work set out to demonstrate: a weakness that can be triggered, a control that addresses it, a repeat of the same test, and an honest statement of the risk that remains. Four of the five attacks are closed by role-based retrieval, data-and-instruction separation, and output cleaning. The fifth stays open and shows why some controls have to live at ingestion rather than at answer time. The method, planting secrets and counting how often they escape, gives a clear number for each claim and makes the before-and-after comparison easy to read.

Environment and reproducibility

The lab is a single Jupyter notebook that runs top to bottom. It talks to a local Ollama server for the language model, the embedding model, the safety classifier, and the judge model, and to a local Weaviate server for the document store. PyRIT runs the attacks and records the results. The notebook saves one row per attempt to a file, so the tables in this report can be rebuilt from the raw data. The embedding model was added to the local Ollama because the chat models do not produce embeddings, and the lab uses Weaviate over its REST interface because the installed client does not support the older server version. Both choices keep the lab self-contained.

References

- OWASP Top 10 for Large Language Model Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/
- MITRE ATLAS, adversary tactics and techniques for AI systems. https://atlas.mitre.org/
- PyRIT, Python Risk Identification Tool for generative AI. https://github.com/Azure/PyRIT